

COMUNICAÇÃO EM SISTEMAS AUTÔNOMOS CRÍTICOS

P1 Grupo M2



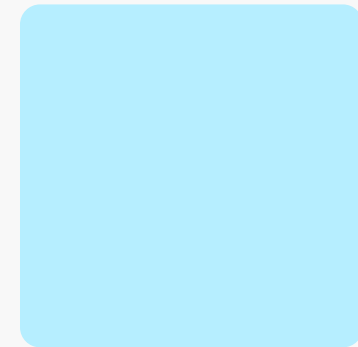
30 de março de
2026

OBJETIVO ATUAL:

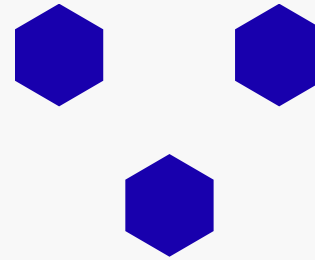
→ Extender a pilha de comunicação para **suportar a troca de mensagens entre os componentes internos de um mesmo veículo.**

- Uma **segunda Engine baseada em memória compartilhada** IPC System V (ShmEngine) é adicionada como único ponto de variação (toda a pilha acima permanece inalterada).
- O **Gateway passa a operar com dois NICs**, fazendo a ponte entre a rede externa e o barramento interno da VM.
- As mensagens agora carregam o **endereço de origem, implementando $M = \{address, payload\}$**

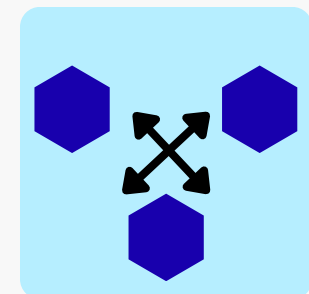
VISUALIZAÇÃO BASE



Cada sistema autônomo é um macro objeto que tem VM QEMU associada

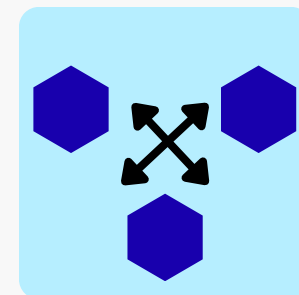


Cada componente é um macro objeto que tem associado a si um Processo dentro da VM



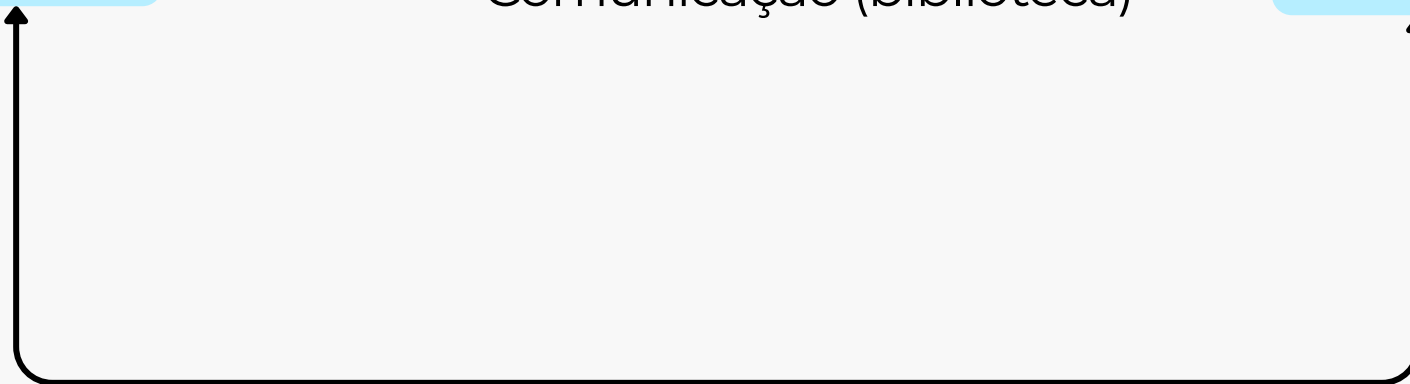
[VM - Veículo A]

- |—— Sensor (processo)
- |—— Controle (processo)
- |—— Comunicação (biblioteca)



[VM - Veículo B]

- |—— Sensor (processo)
- |—— Controle (processo)
- |—— Comunicação (biblioteca)



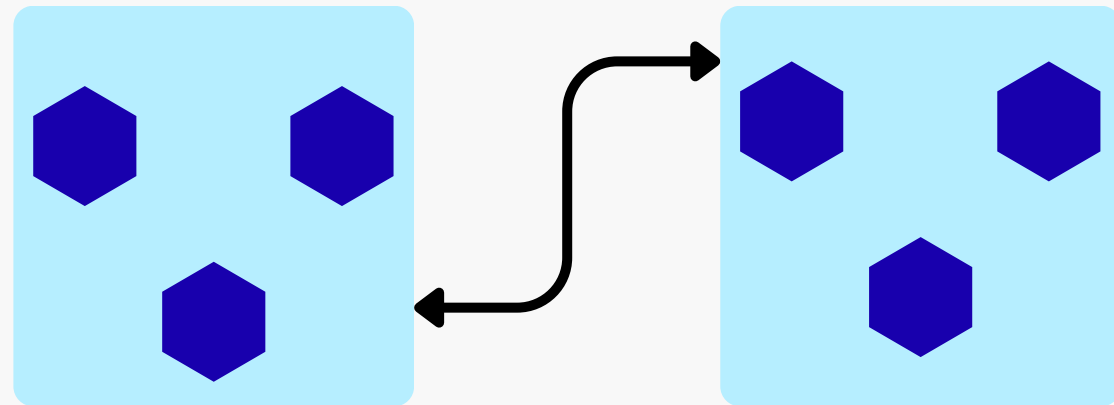
Rede de comunicação

OS 2 DOMINIOS DE COMUNICAÇÃO

Inter-VM

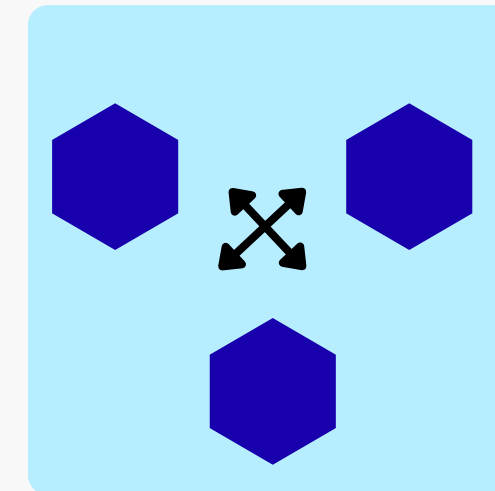
Sistemas conectados em uma rede virtual para comunicação

Comunicação via raw socket, Ethernet virtual



Intra-VM

Componentes se comunicam entre si usando a mesma API via memória compartilhada



Arquitetura em camadas

Componente

Communicator

Protocol

NIC<Engine>

RawSocketEngine

ShmEngine

← **Ator do sistema.** Cada processo (Sensor, Actuator, Gateway..) instancia um Communicator e interage exclusivamente através dele.

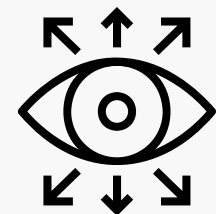
← **Ponto de acesso da aplicação** (interface de acesso para componentes/sensores). Oferece send() e receive() bloqueante. O receive() suspende a thread até que uma mensagem chegue, sem consumir CPU.

← **Multiplexa a NIC por porta.** Monta e desmonta o cabeçalho da mensagem (src, dst, portas, tamanho do payload). Entrega o frame ao Communicator correto com base na porta de destino.

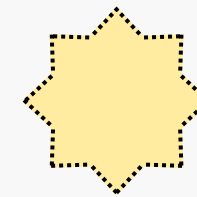
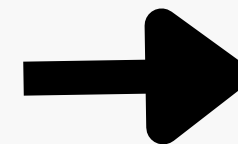
← **Gerencia o pool de buffers de frames Ethernet.** Ao receber um frame, notifica os observadores registrados filtrando pelo EtherType. Instanciada via template com a Engine concreta.

← **abstração do meio físico**
(raw socket para rede Ethernet ou shared memory para IPC)

PADRÃO OBSERVER-OBSERVED COM CONDICIONAL



objetos se registram para serem **notificados** quando algo acontece, sem que o notificador precise conhecê-los diretamente.



Acoplamento fraco (loose coupling)

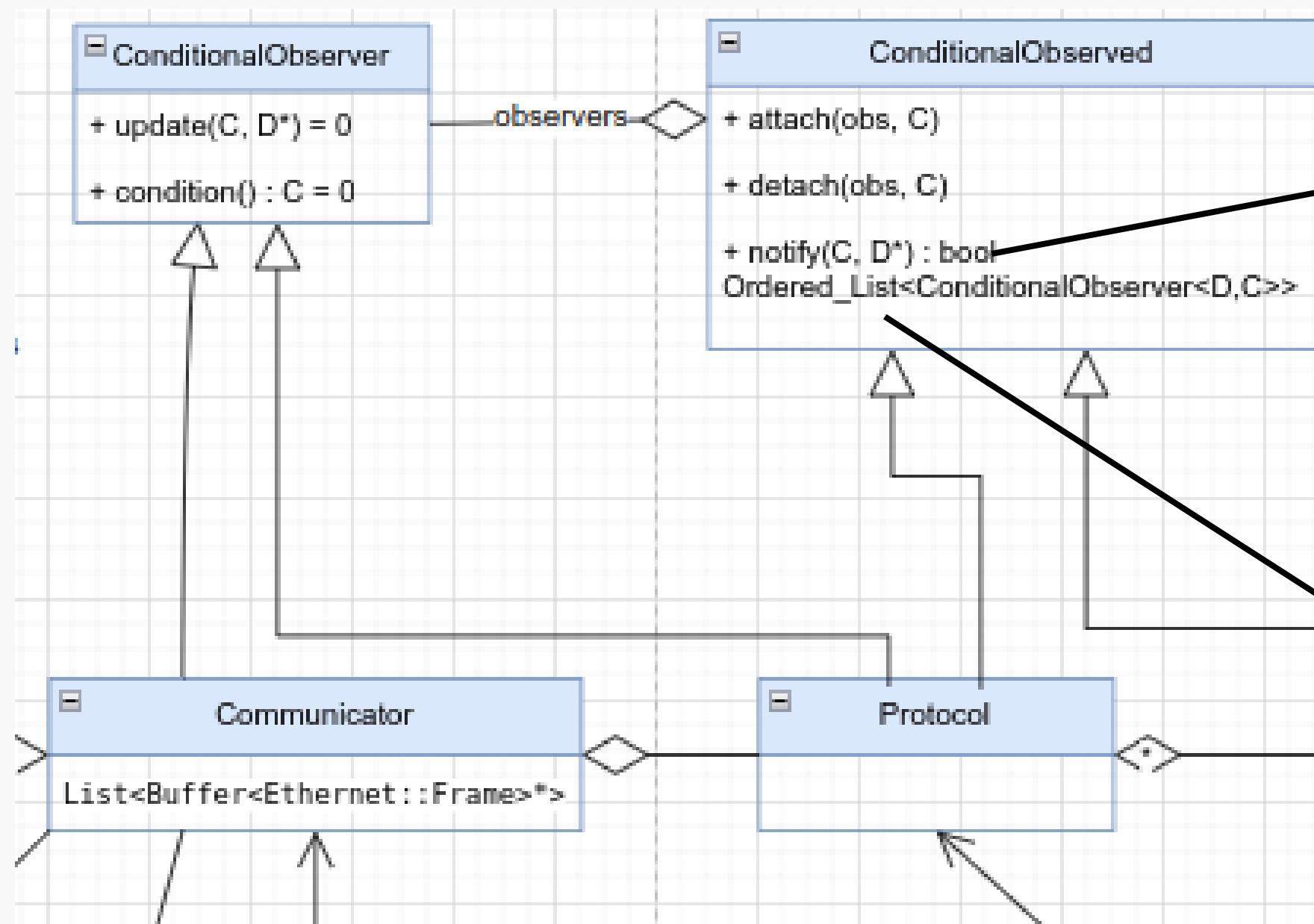
Condicional:

Cada **observer** se registra com uma **condição (C)**, e só é notificado quando aquela condição for satisfeita.

PADRÃO OBSERVER-OBSERVED COM CONDICIONAL NO PROJETO

Por que foi escolhido:

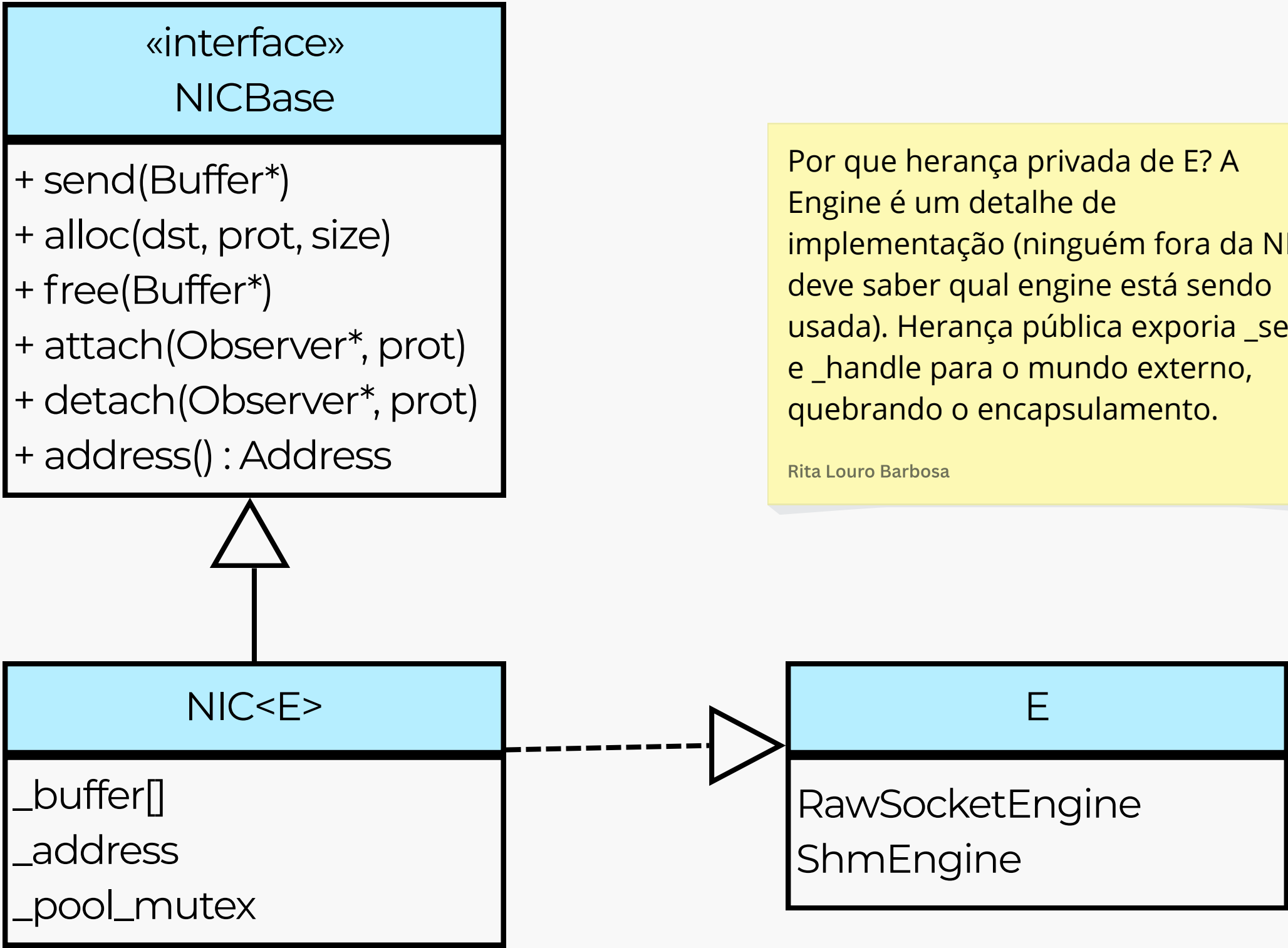
- Recepção assíncrona,
- Desacoplamento entre produtor e consumidor



`notify()` percorre a `Ordered_List` de observers e chama `update()` apenas nos que têm a condição correta (filtragem acontece na própria lista, ordenada pela condição)

Contém lista de observers

NIC<E> : HERANÇA DE TEMPLATE



Por que herança privada de E? A Engine é um detalhe de implementação (ninguém fora da NIC deve saber qual engine está sendo usada). Herança pública exporia `_send` e `_handle` para o mundo externo, quebrando o encapsulamento.

Rita Louro Barbosa

FORMATO DA MENSAGEM NA ETAPA 2

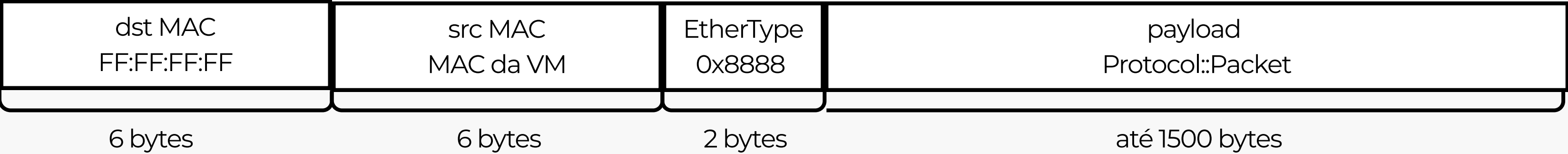
M = {address, payload}

- Pontos-chave:
- O destino Ethernet é sempre broadcast (FF:FF:FF:FF:FF:FF): a filtragem por destinatário acontece nas camadas acima
 - O Protocol::Header implementa o address de M = {address, payload}: contém MAC + porta de origem e destino
 - __attribute__((packed)) garante que não há padding entre os campos (crítico para serialização correta sobre a rede)

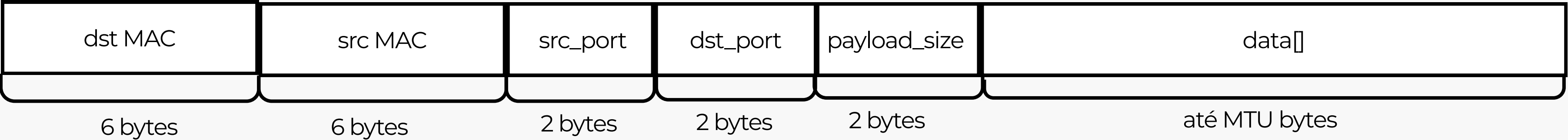
Rita Louro Barbosa

→ Na Etapa 1, o payload era um array de bytes puro.
Na Etapa 2, a mensagem passa a carregar explicitamente a origem, para que o receptor saiba quem enviou e possa responder.

Nível 1 : Ethernet::Frame (enlace)



Nível 2 : Protocol::Packet (dentro do payload Ethernet)



ENDEREÇAMENTO: IDENTIFICANDO CADA COMPONENTE

Protocol::Address combina dois identificadores em um endereço completo:

```
cpp
struct Address {
    Physical_Address paddr; // MAC da VM - 6 bytes
    Port port; // identificador do componente - uint16_t
};
```

Componente	Porta
Gateway	1000
Sensor	1001
Actuator	1002
Powertrain	1003

- Portas são constantes estáticas definidas em ports.hpp

Broadcast e filtragem:

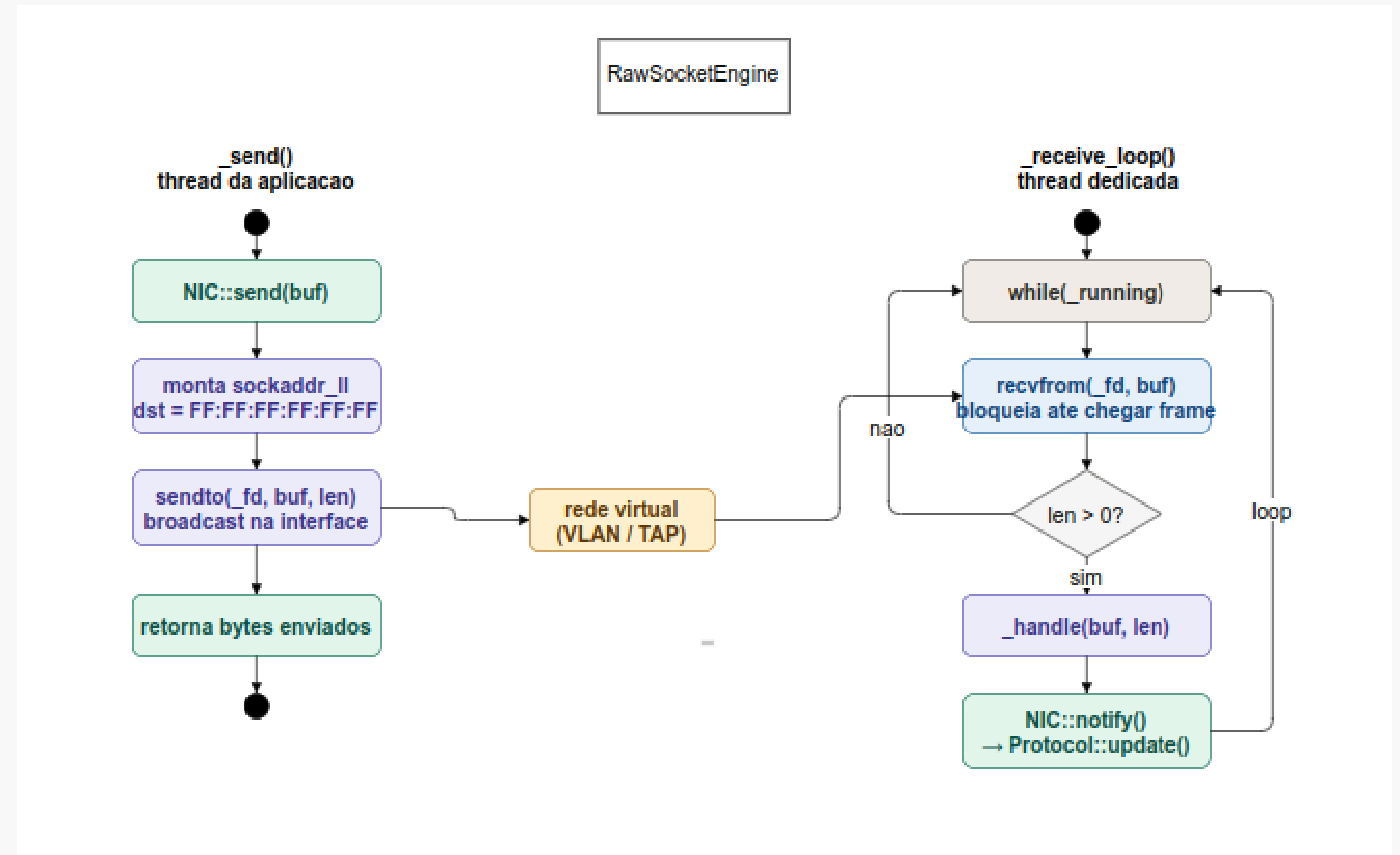
- Nível Ethernet: todos recebem o frame (broadcast físico)
- Nível NIC: filtra por EtherType (0x0800) (só chega ao Protocol)
- Nível Protocol: filtra por dst_port (só acorda o Communicator correto)



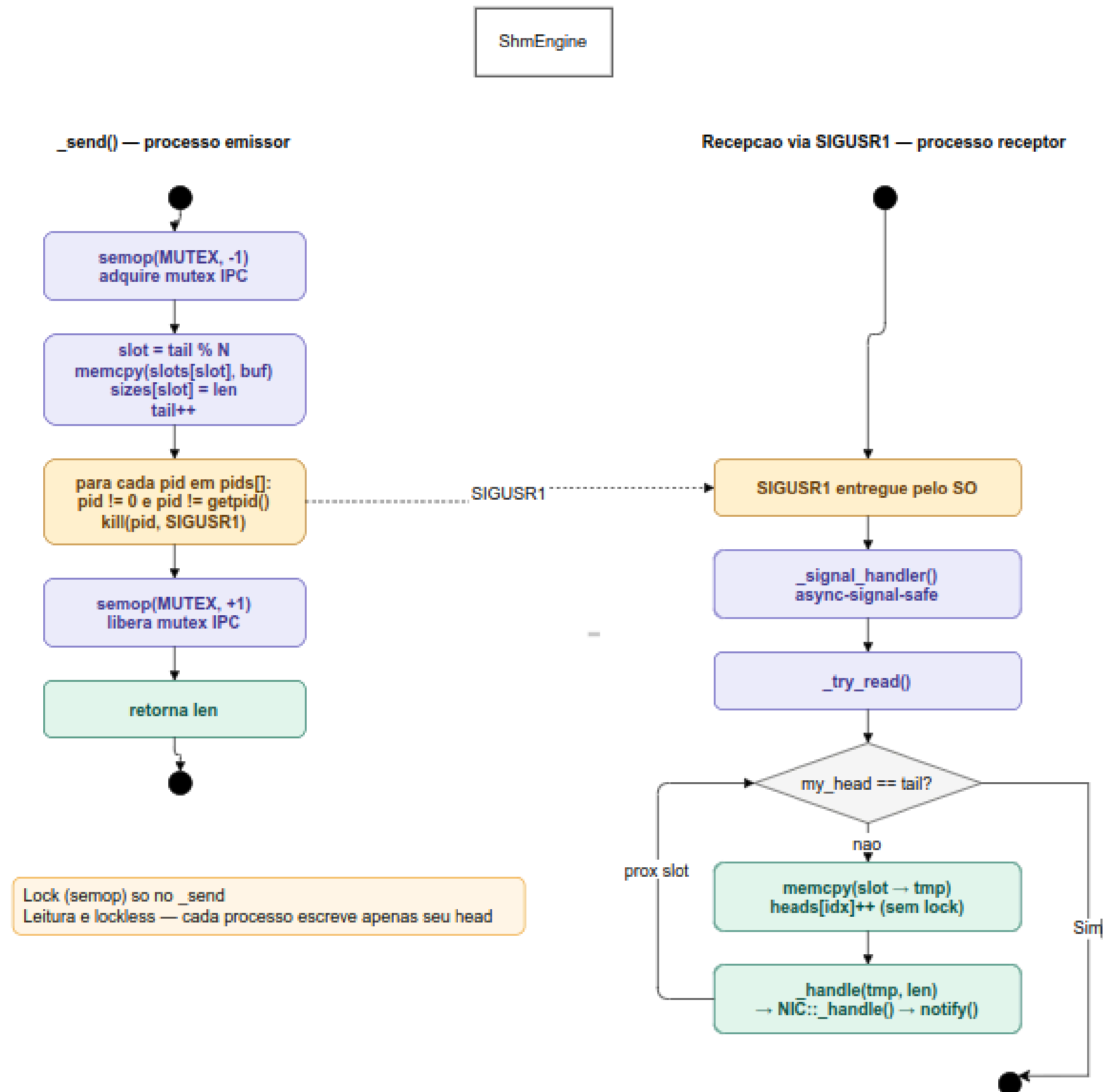
RAW SOCKET ENGINE : COMUNICAÇÃO ENTRE VEÍCULOS

→ Usada pelo Gateway para comunicação entre VMs distintas.

- Opera no nível de enlace (envia e recebe frames Ethernet brutos sem passar pela pilha IP do sistema operacional).



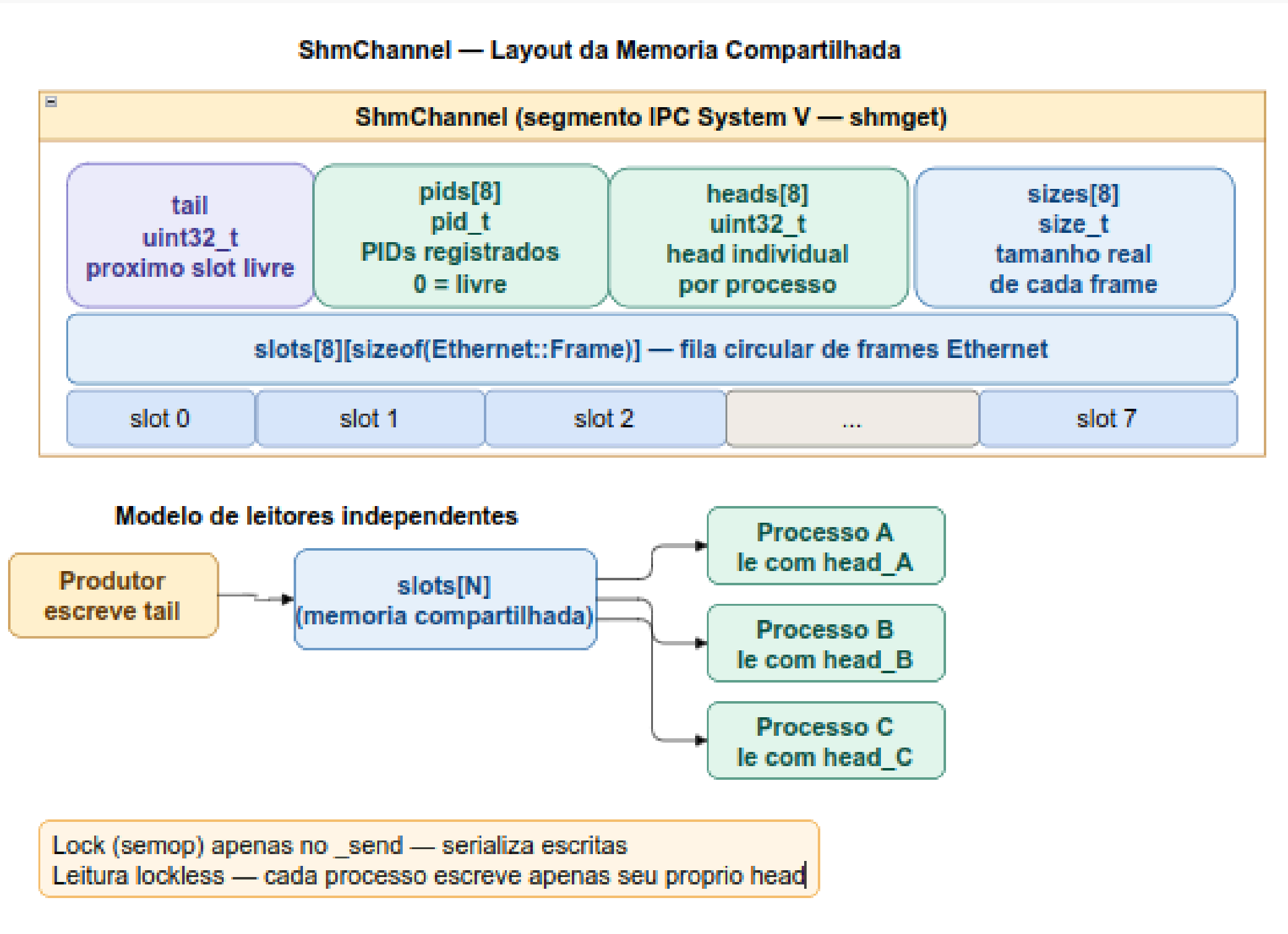
SHMENGINE : COMUNICAÇÃO ENTRE COMPONENTES DO MESMO VEÍCULO



→ Usada pelo Gateway e componenetes para comunicação entre componentes de um mesmo veículo.

- Utiliza memória compartilhada (IPC System V)
- Acesso controlado por semáforos e signal (SIGUSR1)

SHMCHANNEL: LAYOUT DA MEMÓRIA COMPARTILHADA

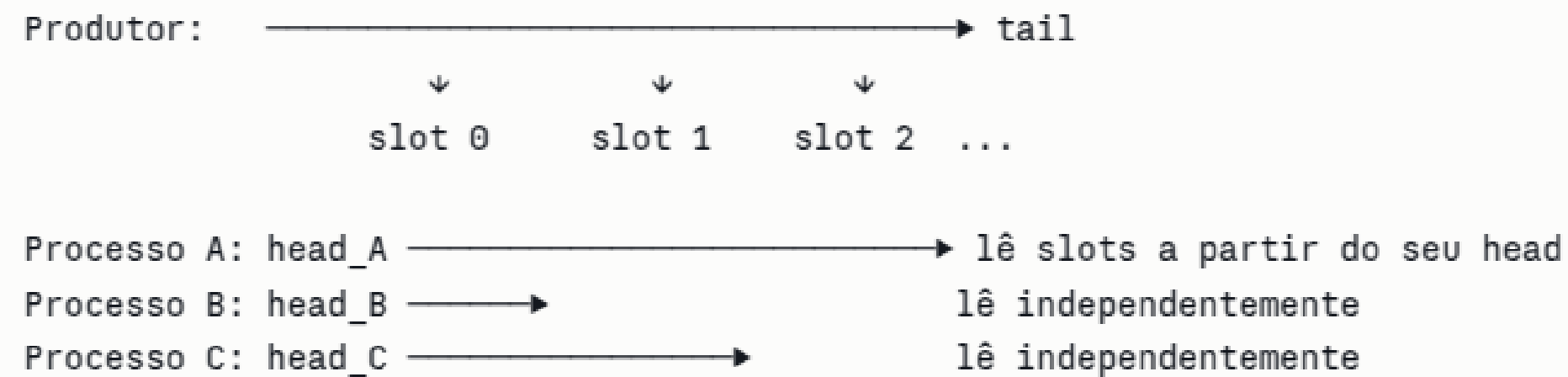


O ShmChannel é a estrutura de dados que vive fisicamente dentro do segmento de memória compartilhada — é o "barramento" que todos os processos da VM enxergam ao mesmo tempo.

- **tail**: ponteiro de escrita
- **heads[]**: cada processo registrado tem seu ponteiro de leitura independente
- **slots[]**: buffers circulares onde os frames ficam até todos os leitores registrados terem passado por eles
- **pids[]**: registro dos pids dos processos para ativar sinal após envio

SHMCHANNEL: LAYOUT DA MEMÓRIA COMPARTILHADA

Modelo de múltiplos leitores:



Decisões de sincronização:

- **Lock no _send:** mutex IPC (semop) serializa escritas concorrentes de múltiplos produtores
- **Sem lock na leitura:** cada processo escreve apenas seu próprio head (não há condição de corrida). O tail é lido com `__atomic_load` para garantir visibilidade sem lock
- **Distinção importante:** semget/semop sincroniza processos (cruzam espaços de endereçamento); `std::mutex` só funciona dentro de um único processo

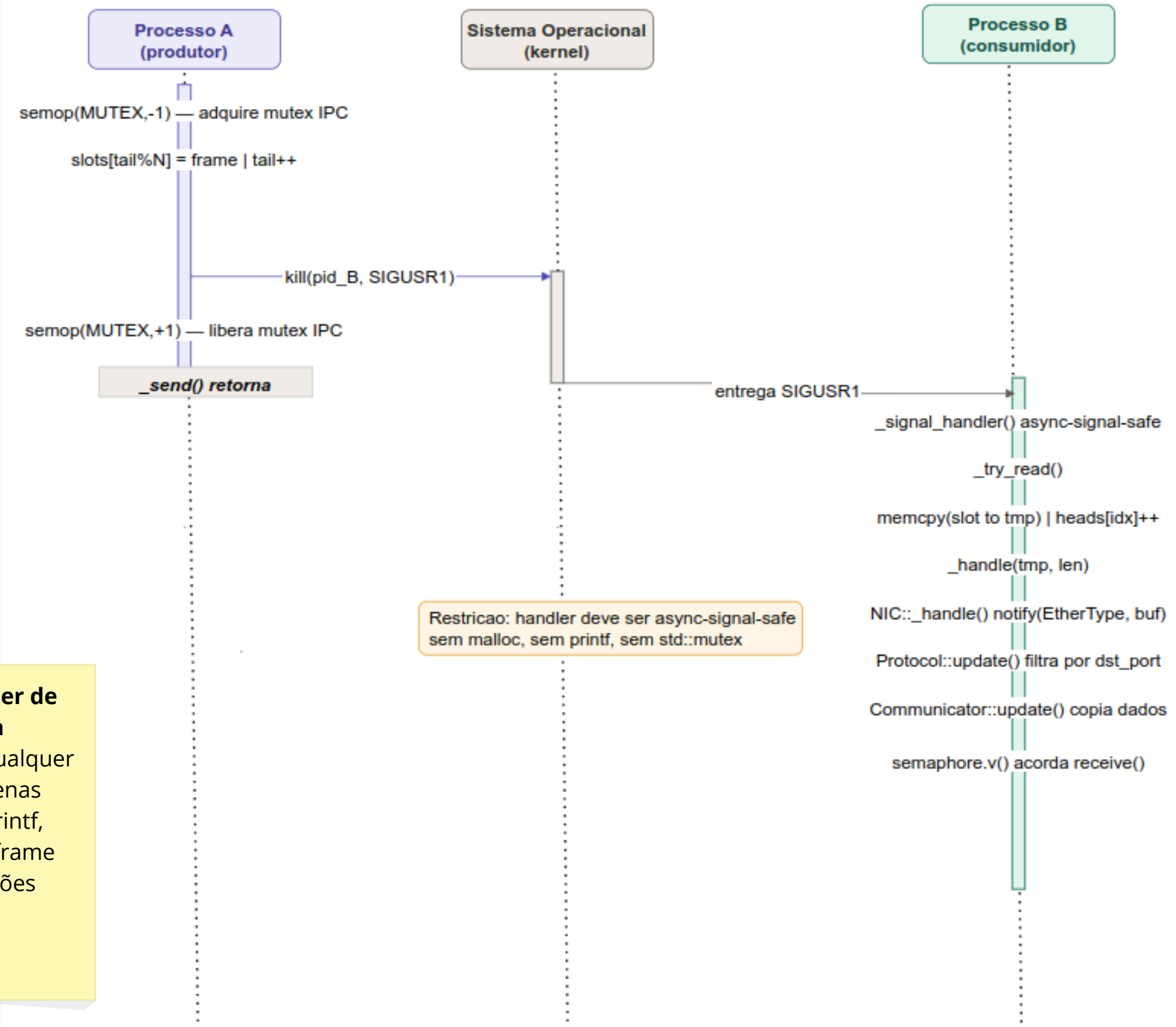
NOTIFICAÇÃO POR SIGUSR1 : DO PRODUTOR AO CONSUMIDOR

- Quando um processo escreve um frame na memória compartilhada, ele precisa avisar os leitores. Em vez de polling ou semáforo bloqueante, **a ShmEngine usa kill(pid, SIGUSR1)**
 - O SO entrega o sinal imediatamente ao processo destino, que processa o frame no handler.

Restrição crítica: async-signal-safety: **O handler de sinal (_signal_handler) é chamado de forma assíncrona pelo SO**, podendo interromper qualquer ponto da execução. Por isso ele deve usar apenas funções async-signal-safe: sem malloc, sem printf, sem locks de std::mutex. O _try_read copia o frame com memcpy e avança o head (ambas operações seguras nesse contexto).

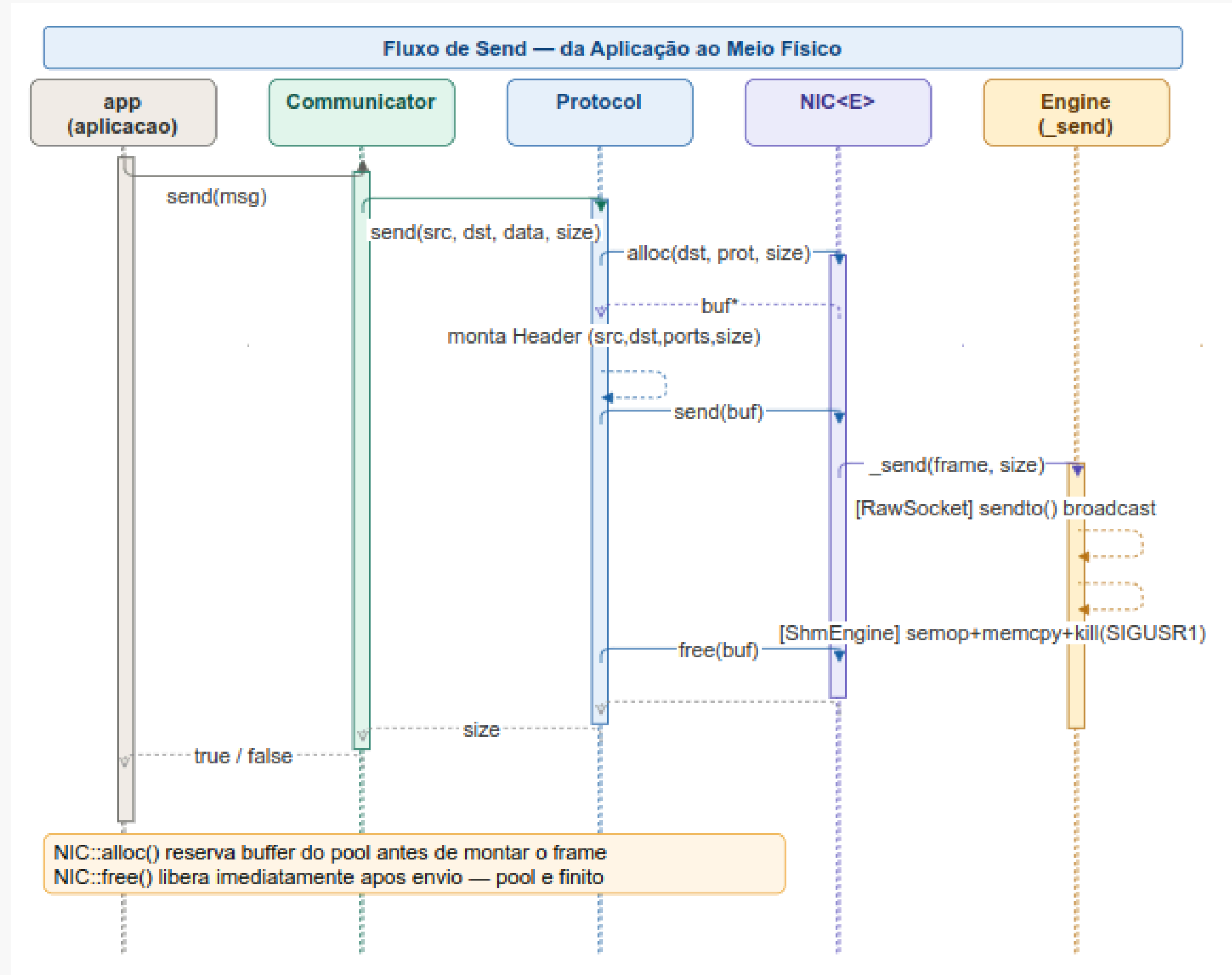
Rita Louro Barbosa

Notificação por SIGUSR1 Do Produtor ao Consumidor



FLUXO DE SEND: DA APLICAÇÃO AO MEIO FÍSICO

- Quando a aplicação chama `send()`, a mensagem percorre toda a pilha de cima para baixo até o meio físico. O caminho é idêntico independente da Engine. A única diferença ocorre na última camada.



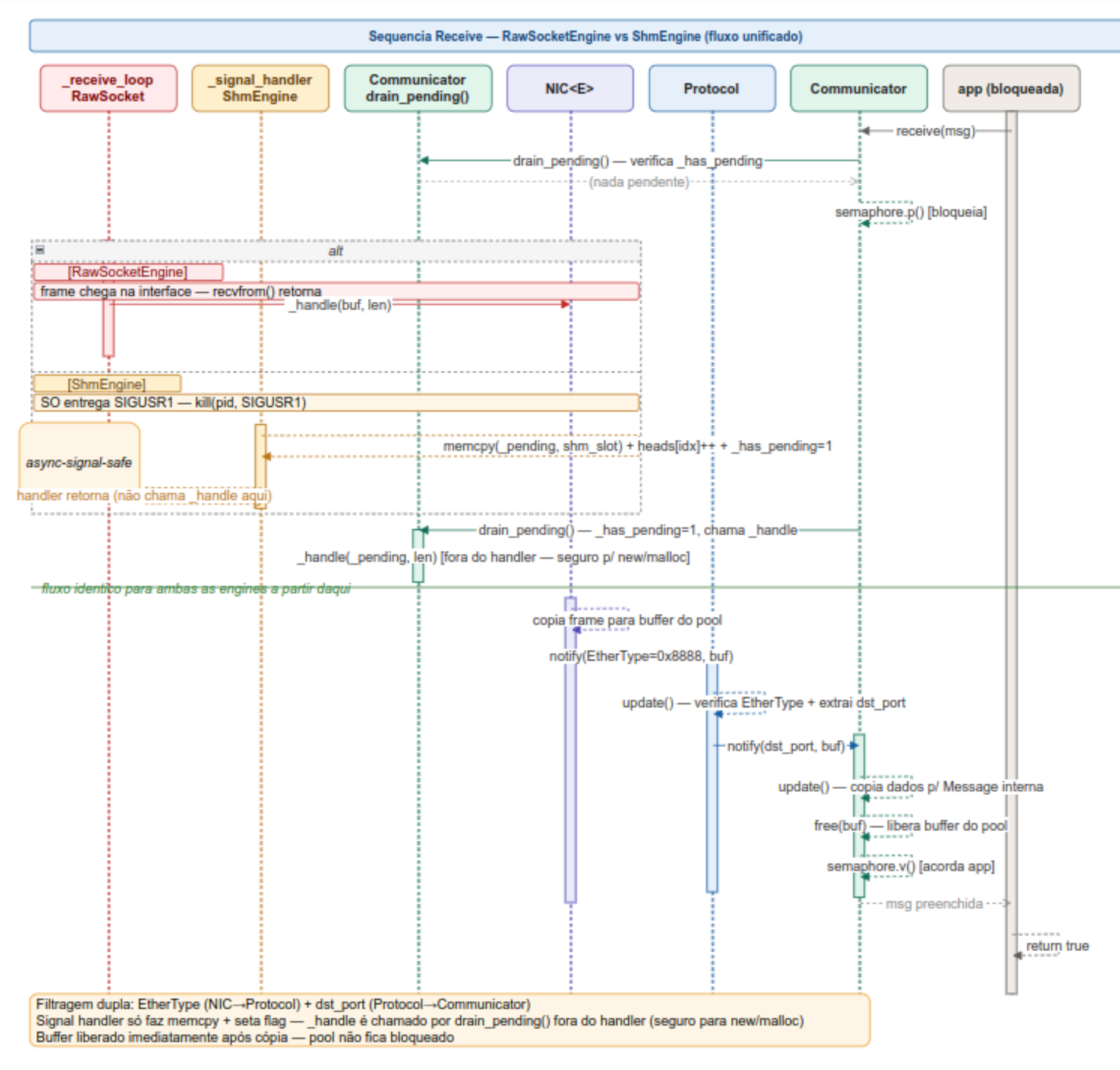
FLUXO DE RECEBE: DO MEIO FISICO ATE A APLICACAO

O caminho de recepção é assíncrono e orientado a eventos. A thread da aplicação fica bloqueada em receive() e só acorda quando um frame chega e percorre toda a pilha de baixo para cima.

Pontos-chave:

- A thread da aplicação já estava bloqueada em semaphore.p() antes do frame chegar
- Communicator::update() copia os dados do buffer da NIC para uma Message interna e libera o buffer imediatamente (o pool não fica travado)
- A filtragem acontece em dois níveis: EtherType na NIC→Protocol, porta no Protocol→Communicator

	RawSocketEngine	ShmEngine
Quem entrega o frame	Thread <code>_receive_loop</code> bloqueada em <code>recvfrom()</code>	SO entrega <code>SIGUSR1</code> , que dispara <code>_signal_handler</code> → <code>_try_read()</code>
Mecanismo de espera	Thread bloqueada — <code>recvfrom</code> suspende até o kernel receber pacote	Processo suspenso em qualquer ponto — sinal interrompe e executa o handler
Lectura do dado	Kernel copia direto para o buffer via <code>recvfrom</code>	<code>_try_read</code> lê da memória compartilhada de forma lockless
A partir de	Idêntico	Idêntico
	<code>_handle()</code>	

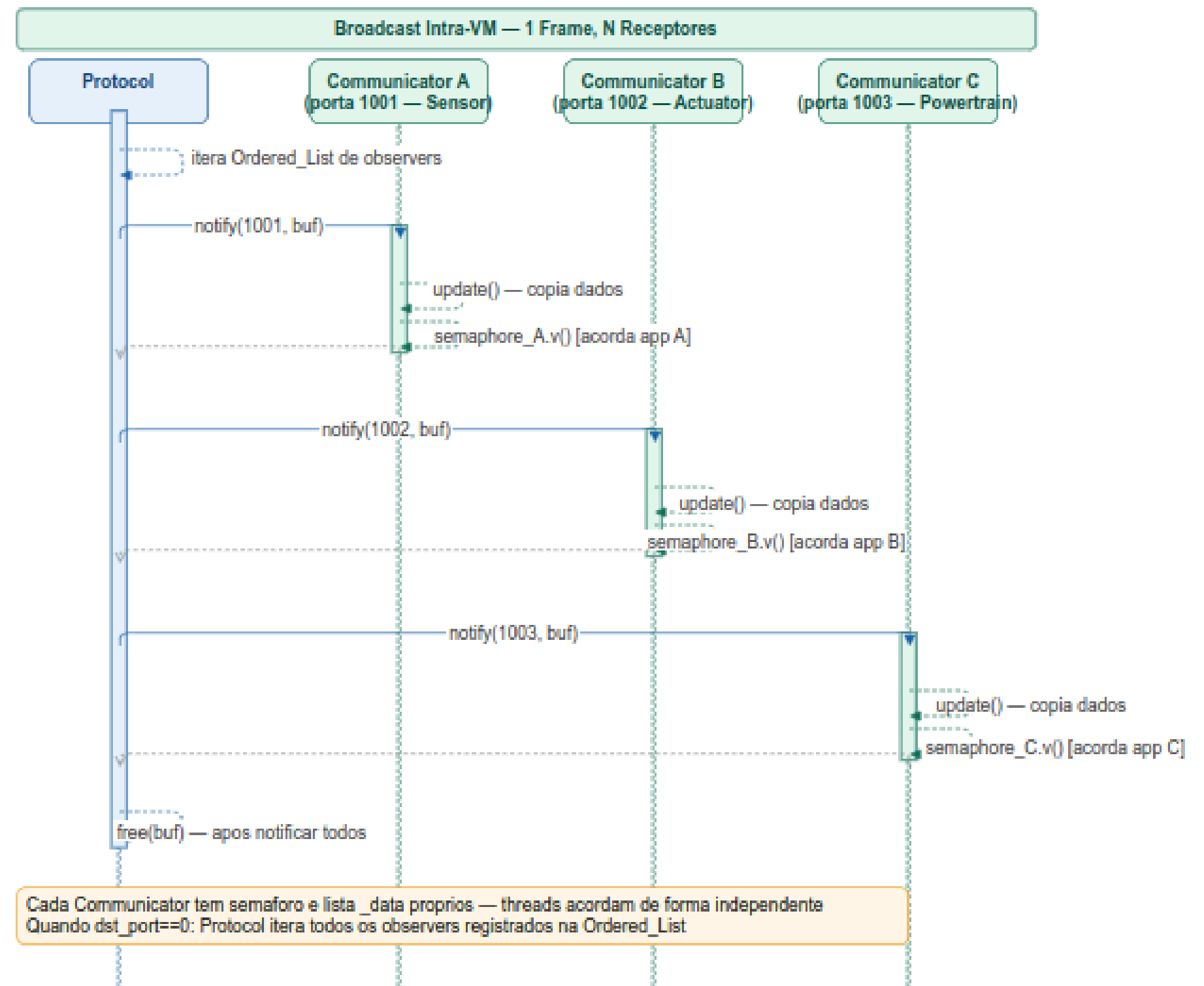


BROADCAST INTRA-VM

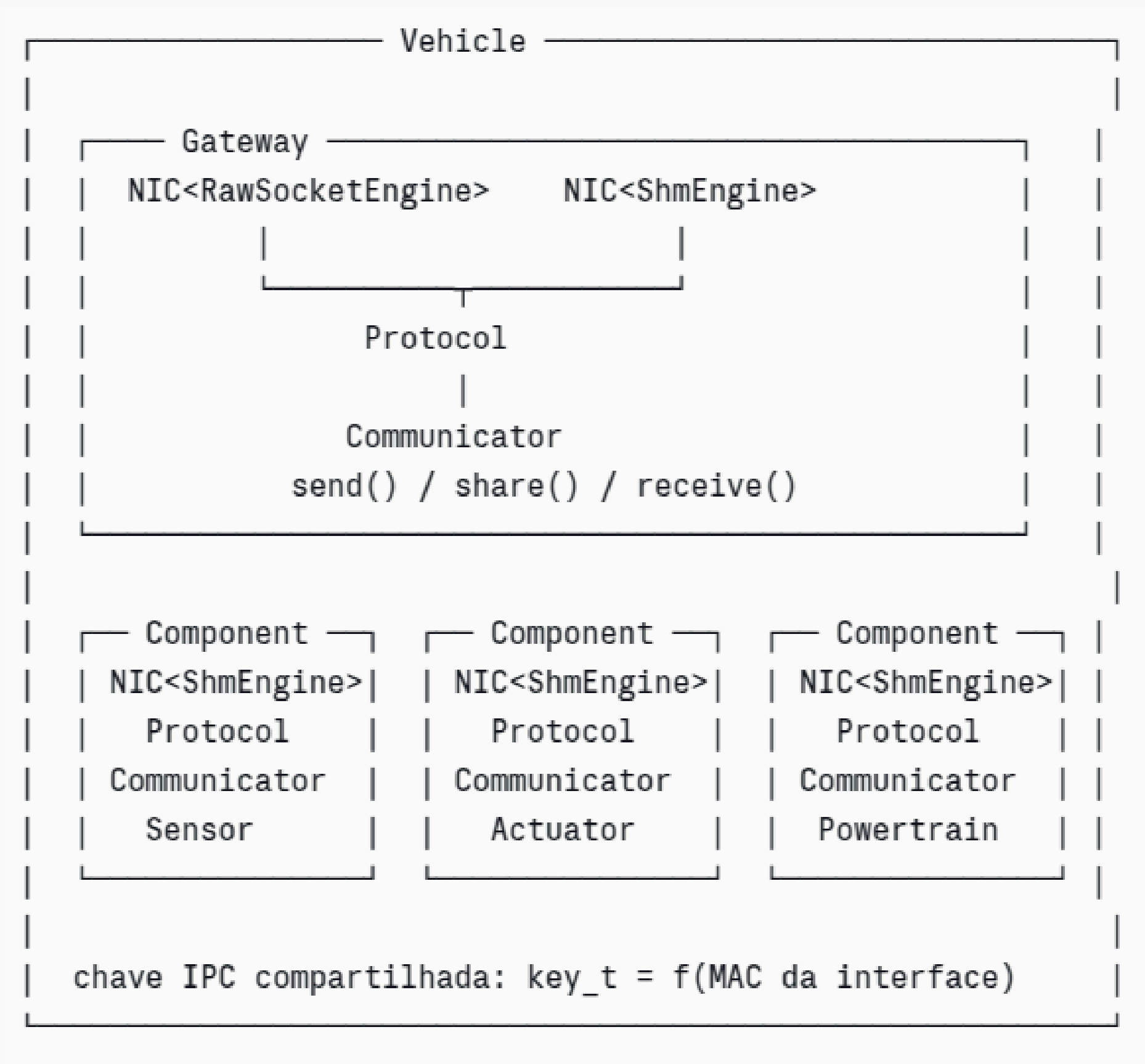
: Como o envio é sempre em broadcast, todos os componentes registrados recebem o mesmo frame. Cada um acorda de forma independente — não há coordenação entre eles.

Pontos-chave:

- Protocol::update() itera a Ordered_List de observers e notifica cada porta registrada (no código atual, quando `dst_port == 0` (broadcast), notifica todas as portas conhecidas)
- Cada Communicator tem seu próprio semáforo e lista `_data` — não compartilham estado
- Thread-safety garantida: `List<T>` tem mutex interno, semáforo é por instância
- As threads da aplicação acordam de forma independente e concorrente e não há serialização entre elas



ESQUEMÁTICO DO VEÍCULO



Ponto-chave: O Protocol do Gateway recebe frames de ambos os NICs (identifica pela porta de destino para qual Communicator entregar.) A decisão de usar send() ou share() é da camada de aplicação, não da pilha.

TESTE DE LATÊNCIA

```
[rtt] amostra=1666 rtt=2022us media=3061us
[rtt] amostra=1667 rtt=4462us media=3062us
[rtt] amostra=1668 rtt=3251us media=3062us
[rtt] amostra=1669 rtt=3548us media=3062us
[rtt] amostra=1670 rtt=3790us media=3062us
[rtt] amostra=1671 rtt=3764us media=3063us
[rtt] amostra=1672 rtt=3189us media=3063us
[rtt] amostra=1673 rtt=3569us media=3063us
[rtt] amostra=1674 rtt=3780us media=3064us
[rtt] amostra=1675 rtt=3604us media=3064us
[rtt] amostra=1676 rtt=4009us media=3065us
[rtt] amostra=1677 rtt=3288us media=3065us
[rtt] amostra=1678 rtt=3827us media=3065us
[rtt] amostra=1679 rtt=3937us media=3066us
[rtt] amostra=1680 rtt=3582us media=3066us
[rtt] amostra=1681 rtt=3965us media=3067us
[rtt] amostra=1682 rtt=4324us media=3067us
[rtt] amostra=1683 rtt=4123us media=3068us
[rtt] amostra=1684 rtt=4184us media=3069us
[rtt] amostra=1685 rtt=3191us media=3069us
[rtt] amostra=1686 rtt=4069us media=3069us
```

□